Vision - module

Technical documentation

Mikołaj Baran, Kewin Kisiel, Szymon Wójcik

March 2026

Contents

1 A word of introduction

The primary goal of the project is to construct a robotic head with anthropomorphic features, capable of two-way communication with the user and the expression of three selected emotional states. The entire system has been divided into cooperating modules. The scope of work for the described team covers the design and implementation of the visual perception module (the robot's vision system). The main task of this module is to detect the user's face in a video stream and extract its characteristic points (facial landmarks). The data obtained in this way will be used to build a profile database, enabling the system to identify returning individuals and initiate personalized interaction (e.g., greeting the user by name). Another key element of the vision module is the implementation of a classifier that recognizes the user's affective states. The algorithm is intended to recognize seven emotion classes: anger, sadness, fear, surprise, disgust, happiness, and a neutral state. In the final phase of model design and optimization, a classification accuracy of approximately 75 percent is assumed. The output data from the vision module (identified identity and recognized emotion) will be passed to the speech synthesis subsystem. This will enable dynamic adjustment of both the content and acoustic parameters of the robot's speech to the interlocutor's current emotional state.

1.1 Technological Stack

The implementation of the vision module's tasks will be based on the following technologies and algorithmic methods:

- Face Detection - A pre-trained BlazeFace [1] model (developed by Google) will be used for fast and efficient face localization (determination of bounding boxes) in image frames. This model is characterized by high performance, which is critical in real-time operating systems.
- Face Alignment / Landmarking - The MediaPipe [2]framework (Face Mesh / Face Landmarks module) will be applied. This solution was chosen due to its high precision, optimization for real-time operation, and significantly greater reliability compared to classical, custom-built vision algorithms.
- User Identification (Face Recognition) - The identity verification process will be implemented using deep neural networks. An aligned (cropped based on facial landmarks) face image will be processed by the model to generate a feature vector (embedding). Subsequently, user recognition will be performed by computing the cosine similarity between the generated vector and reference vectors stored in the system's database.

2 Dataset preprocessing

2.1 Search for datasets

The process of acquiring data sets encountered limitations resulting from licensing restrictions, making it impossible to use them for design purposes.Resource exploration was carried out within key repositories and analytics platforms, such as GitHub, Kaggle and Hugging Face.The analyzed datasets often aggregated popular datasets such as FER-2013, RAF-DB, and AffectNet, supplemented with resources obtained from Google Images and unknown sources. The process of verifying and correctly classifying them required a multi-stage analysis, combining manual methods with algorithmic detection of duplicates. The data set from the Hugging Face platform, generated synthetically using the Gemini 2.0 model, proved to be a particular challenge. This collection was characterized by low annotation quality, and analysis of related JSON files revealed inconsistencies in structure and the inability to correctly categorize photos. Ultimately, the data sets compiled in Figure 1 were selected for the project.

Dataset	License	Files	Resolution	Emotions	Source
(fer2013+affectnet) dataset - Emotions	Apache 2.0	61.7 thousand	48x48	7	Kaggle
Facial Affect Dataset Balanced	Attribution-NonCommercial-ShareAlike 3.0 IGO (CC BY-NC-SA 3.0 IGO)	246 thousand	96x96	8	Kaggle
emonet-face-big	Creative Commons Attributio	203 thousand	mixed	no annotation	HuggingFace
Human Face Emotions	Apache 2.0	59.1 thousand	mixed	5	Kaggle
Facial Emotion Recognition Dataset	Attribution 4.0 International (CC BY 4.0)	49.8 thousand	mixed	7	Kaggle

Figure 1: List of used datasets

2.2 Deduplication and augmentation filtering

The first and most drastic step in reducing informational noise was deduplication. A custom script utilizing the perceptual hashing (pHash) algorithm was implemented for this purpose. Unlike standard methods that compare files byte by byte, pHash analyzes the visual structure of the photographs. This allowed for the effective detection and removal of duplicates even in cases where the images differed in resolution, format, or had previously undergone minor artificial augmentation.

To optimize computation time, the hash generation process was parallelized using multithreading. The algorithm continuously cached the unique image signatures. Any new photograph whose generated hash was already in the set was immediately classified as a duplicate and rejected. This step was essential to avoid data redundancy and uncontrolled sample growth. The process was highly

effective, rejecting 83,380 duplicated photographs. A detailed breakdown of the deduplication results for individual emotion classes is presented in Table 1.

Category	Identified (Initial)	Kept	Removed (Duplicates)
Neutral	24054	21888	2166
Happy	48434	26309	22125
Sad	32766	17540	15226
Angry	28749	15326	13423
Fear	28388	15368	13020
Disgust	13437	9799	3638
Surprise	26732	12950	13782
Total	**202560**	**119180**	**83380**

Table 1: Summary of the deduplication process by emotion category

2.3 Face detection and demographic filtering

Due to the presence of shots in the raw dataset that did not contain human faces (e.g., inanimate objects, road signs), the dataset was passed through a face detection algorithm. Examples of samples rejected at this stage are shown in Figure 2.

Figure 2: List of Bad Photos

Subsequently, the dataset of 119,180 photographs was filtered based on demographic criteria. In accordance with the project's assumptions, images depicting children's faces were removed. This filtration phase reduced the dataset by 7,143 samples, leaving a base of 112,037 images for further processing.

2.4 Artifact removal and manual visual verification

In the penultimate phase, a dedicated algorithm was implemented to remove text overlays and watermarks from the images. The EasyOCR library with GPU hardware acceleration was utilized for this task. The tool scanned each image for alphanumeric characters. If a text string consisting of more than two letters was detected, the system automatically classified the sample as contaminated

and moved it to a rejection folder. This eliminated informational noise that could disrupt the neural network's extraction of key features.

The automated process was supplemented by the most time-consuming stage: manual inspection of the data. The combined efforts of the OCR algorithm and manual review allowed for the exclusion of animated graphics, drawings, and shots characterized by partial occlusion of the face, which prevented accurate analysis of facial expressions. This resulted in a total of 7,840 defective samples being removed. Following these steps, the dataset was reduced to 104,197 images.

2.5 Annotation correction

The verification revealed numerous errors in the original class labels, resulting from the automated aggregation of data from various sources. Incorrectly classified images were re-annotated. Samples with ambiguous expressions that could not be unambiguously assigned to a specific emotion were permanently removed from the dataset. Examples of original misclassification are illustrated in Figure 3 (default labels, from left: Fear, Sadness, Neutral).

Figure 3: List of bad annotated photos

After completing all the aforementioned cleaning, filtering, and correction steps, the final dataset ready for model training stabilized at the level of 104,197 selected and normalized images.

3 Final training dataset

4 Convolutional neural networks

Convolutional neural networks (CNNs) are the most prominent type of AI algorithm for image processing out there today. Put simply, a CNN takes a visual input (a digital image) and applies a series of filter operations to that input. Through this process, a certain type of information or output is extracted from the input image. This could, for example, be the type of object that is displayed in the

input (a.k.a. image classification), but it could also be a partitioning of the input into different regions (a.k.a. image segmentation) or something else entirely. CNNs are very versatile in that respect and can be used to solve all kinds of image processing problems. For those interested in analogies with biology, think of the way that the visual cortex processes information (cf. works of Hubel & Wiesel) – CNNs are inspired by and closely related to this architecture.[1]

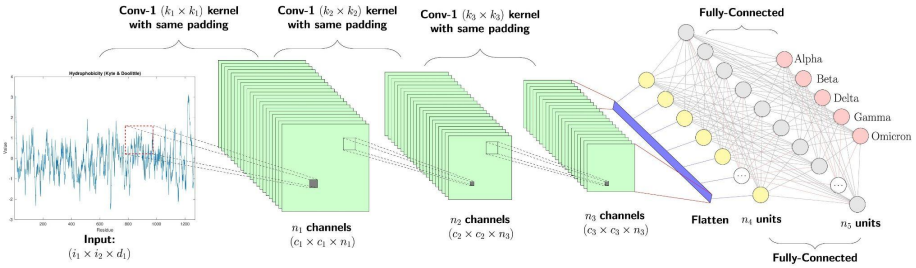

Figure 4: CNN network example

4.1 Literature Review and Theoretical Foundation

The development of an original neural network architecture was preceded by a thorough review of the relevant literature and scientific publications in the field of human emotion recognition using Convolutional Neural Networks (CNNs). Knowledge regarding the operational mechanisms of convolutional networks and the functions of individual model layers was systematized based on the official technical documentation of the TensorFlow [3] library and specialist educational materials (including the GeeksforGeeks portal[2]). The following technology stack was employed in the process of designing and implementing the system:

- TensorFlow — The selection of this framework was driven by its high performance and streamlined deployment process for trained models on embedded platforms. This aspect was of critical importance due to the intended implementation of the system on a Raspberry Pi microcomputer.
- TensorBoard — This tool was implemented for the purpose of monitoring and evaluating model metrics. It enables real-time logging of parameters during the training process and their visualization within a dedicated graphical interface, while offering advanced filtering and data view customization capabilities.

[1] Mateusz Miłosz, *Convolutional Neural Networks Demystified: How Immunofluorescence Images Are Processed in Modern Labs.* Available at: [Link to the article]

[2] shibsankar saw, *Emotion Detection Using Convolutional Neural Networks (CNNs).* Available at: [Link to the article]

- TensorFlow Keras (tf.keras) — This module was used to optimize dataset management. It facilitated the partitioning of the available corpus into training, validation, and test sets, in proportions of 70 percent 20 percent 10 percent respectively. Furthermore, due to significant class imbalance within the dataset (a considerably smaller number of samples for selected emotion categories), the library's mechanisms were employed to analytically compute and assign weights to individual classes, thereby preventing the model from exhibiting a bias toward dominant classes.
- Scikit-learn (sklearn) — This library was utilized primarily for the generation of a confusion matrix. It constitutes a fundamental analytical tool within the project, enabling detailed evaluation of classifier performance and assessment of prediction accuracy across individual emotion categories.

4.2 Problems encountered

4.2.1 Environmental Challenges and Hardware Resource Optimization

During the implementation phase of the neural network model, several critical issues of an environmental and hardware nature were identified. The main obstacle was the lack of native GPU acceleration support on Windows for the TensorFlow library (version 2.11 and later). This problem was resolved by migrating the runtime environment to the WSL2 subsystem (Windows Subsystem for Linux). A further challenge was the aggressive allocation of all available GPU VRAM by TensorFlow processes, which led to system instability. This issue was mitigated at the algorithm code level by enabling dynamic memory management (memory growth). As a result, only the amount of VRAM strictly necessary for efficient computation at any given moment is allocated to the network training process.

4.2.2 Overfitting and the Model Checkpointing Mechanism

During training runs on various datasets, it was observed that models tend to overfit very rapidly. The critical point at which the model's generalization capability began to deteriorate sharply typically occurred between the 8th and 11th epoch. To prevent the loss of the best network weights, a Model Checkpointing mechanism was implemented. The algorithm was configured to permanently save the model state only during those epochs in which the validation loss reaches a value lower than in previous iterations.

4.2.3 Analysis of the Root Causes of Early Overfitting

The rapid degradation of validation results and the aggressive overfitting observed in the trained models stems from the convergence of several factors, related primarily to the quality and characteristics of the input data:

- Memorization - Rather than extracting universal features, a model with high parameter capacity begins to "memorize" specific images from the training set. This results in high accuracy on training data and a drastic drop in performance on new, previously unseen images.
- Noisy Labels - The datasets used frequently contain low-quality images ("garbage" data) or carry incorrectly assigned labels (e.g., a neutral face labeled as sad). The network's attempt to fit such anomalies forces it to develop false, overly complex decision rules, which directly accelerates the overfitting process.
- Insufficient Data Diversity - A lack of adequate variance in the training set causes the model to quickly exhaust the pool of available features. In such cases, the application of strong regularization and Data Augmentation techniques becomes essential.

4.3 First simple model

The designed architecture constitutes a classical deep convolutional neural network (CNN), optimized for the extraction of spatial features from facial images and the classification of emotional states. The model was built in a layered fashion, with a clear division into a feature extraction module and a final element serving as the classifier.

4.3.1 Code review

```
def first_model():

    inputs = Input(Input_shape)

    conv1 = layers.Conv2D(32, (3, 3), strides=(1, 1), kernel_regularizer=l2(0.001))(inputs)
    conv1 = layers.Dropout(0.1)(conv1)
    conv1 = layers.Activation('relu')(conv1)
    pool1 = layers.MaxPooling2D((2, 2))(conv1)

    conv2 = layers.Conv2D(64, (3, 3), strides=(1, 1), kernel_regularizer=l2(0.001))(pool1)
    conv2 = layers.Dropout(0.1)(conv2)
    conv2 = layers.Activation('relu')(conv2)
    pool2 = layers.MaxPooling2D((2, 2))(conv2)

    conv3 = layers.Conv2D(128, (3, 3), strides=(1, 1), kernel_regularizer=l2(0.001))(pool2)
    conv3 = layers.Dropout(0.1)(conv3)
    conv3 = layers.Activation('relu')(conv3)
    pool3 = layers.MaxPooling2D((2, 2))(conv3)

    conv4 = layers.Conv2D(256, (3, 3), strides=(1, 1), kernel_regularizer=l2(0.001))(pool3)
    conv4 = layers.Dropout(0.1)(conv4)
```

```
    conv4 = layers.Activation('relu')(conv4)
    pool4 = layers.MaxPooling2D((2, 2))(conv4)

    # Classifier
    flatten = layers.Flatten()(pool4)
    dense = layers.Dense(128, activation='relu')(flatten)
    drop_1 = layers.Dropout(0.2)(dense)
    outputs = layers.Dense(num_emotions, activation='softmax')(drop_1)

    model = Model(inputs=inputs, outputs=outputs, name="first_model")
    model.summary()
    return model
```

- `layers.Conv2D(32, (3, 3), strides=(1, 1), kernel_regularizer=l2(0.001))`
  This is a two-dimensional convolutional layer responsible for extracting features from facial images. It slides 32 different filters of size 3x3 (kernel) across the entire image one pixel at a time (`strides=(1,1)`), searching for edges, curves, and specific textures. The `kernel_regularizer=l2` parameter adds a penalty to the loss function for excessively large weight values, which is a technique used to prevent model overfitting.

- `layers.Dropout(0.1)`
  A regularization layer that randomly "switches off" (zeroes out) a specified fraction of neurons (in this case, 10%) during each training step. This forces the network to develop more generalized patterns and prevents overfitting to specific images in the training set.

- `layers.Activation('relu')`
  Defines the ReLU (Rectified Linear Unit) activation function. Its role is to introduce non-linearity into the model by zeroing out all negative values while passing positive values through unchanged. This enables the network to learn complex, non-linear relationships.

- `layers.MaxPooling2D((2, 2))`
  A pooling layer that reduces the spatial resolution of the feature map by half. It selects the highest value from each 2x2 pixel region. This drastically reduces the number of parameters to be computed and makes the model less sensitive to minor shifts of the face within the frame.

- `layers.Flatten()`
  A flattening layer. It takes the three-dimensional feature maps generated by the preceding convolutional layers and reshapes them into a one-dimensional vector. This is a necessary step in order to pass the extracted features to a classical dense network (the classifier).

- `layers.Dense(128, activation='relu')`
  A fully connected dense layer (every neuron is connected to every neuron in the preceding layer). It acts as the primary classifier, making decisions

based on the flattened feature vector that lead to emotion recognition. It contains 128 neurons and its own ReLU activation function.

- `activation='softmax'`
  An activation function used in the final (output) layer of the network for multi-class problems. It transforms the network's raw numerical outputs into a probability distribution — as a result, the output values always sum to 1.0 (100%).

- `model = Model(inputs=inputs, outputs=outputs, name="first_model")`
  A function from the Keras Functional API that physically creates the model object. It connects the previously defined data entry point (`inputs`) with the final layer (`outputs`) into a single graph structure, which can then be compiled, trained (by calling the `.fit()` method), and used for inference.

4.3.2 Training metrics

The neural network training process was carried out using a dataset comprising 104,197 grayscale images with a standardized resolution of 96×96 pixels. At this stage of experimentation, data augmentation techniques were deliberately omitted in order to establish the model's baseline generalization capability. As a result, upon completion of the training process, a validation accuracy of approximately 63% was achieved. The computational experiments were conducted in a local environment (a mobile workstation), utilizing the Windows Subsystem for Linux (WSL2) architecture to ensure framework compatibility with hardware acceleration. The hardware platform used for training had the following specification:

- CPU - Intel Core i9 13th Generation (base clock 2.20 GHz)
- RAM - 16 GB DDR5 (5600 MHz, CL46 latency)
- GPU - NVIDIA GeForce RTX 4060 with 8 GB dedicated VRAM

Based on the conducted training, we generated the confusion matrix shown in Figure 5.

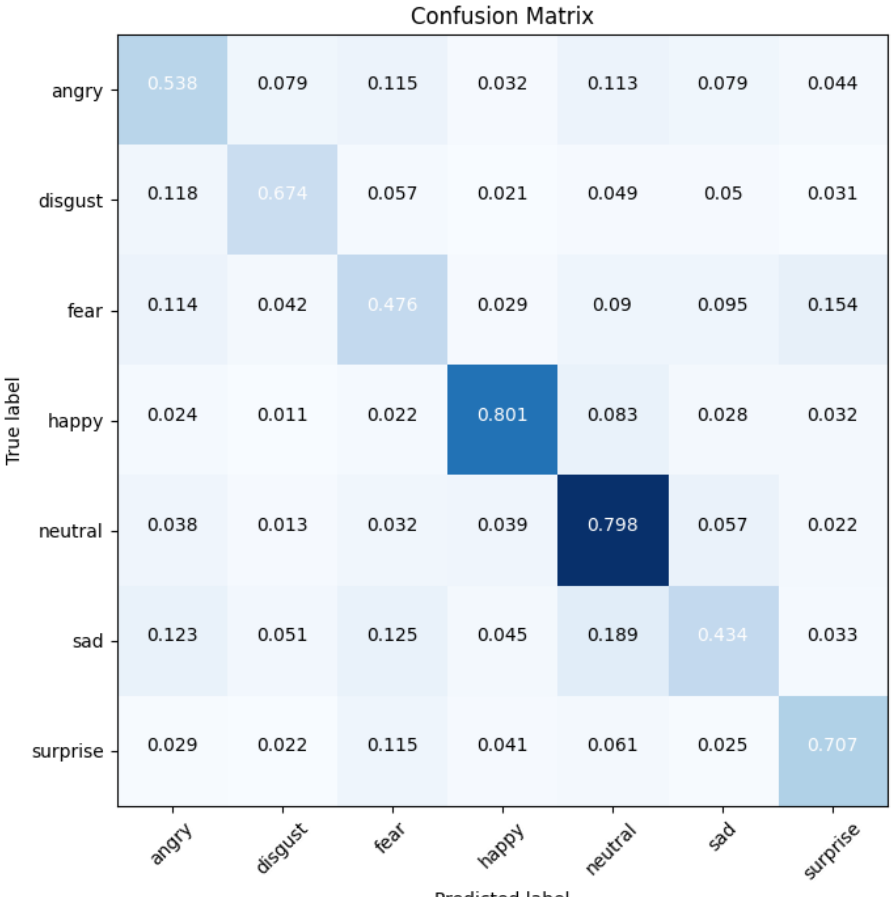

Figure 5: Confusion matrix

- Accuracy

$$\begin{aligned}\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{0.538 + 0.674 + 0.476 + 0.801 + 0.798 + 0.434 + 0.707}{7} \\ &\approx 0.6326 \ (63.26\%)\end{aligned}$$

- Precision

$$\begin{aligned}\text{Precision}_{\text{happy}} &= \frac{TP}{TP+FP} \\ &= \frac{0.801}{0.032+0.021+0.029+0.801+0.039+0.045+0.041} \\ &= \frac{0.801}{1.008} \approx 0.7946 \ (79.46\%)\end{aligned}$$

Analysis of the loss function curves for the training and validation sets (Figure 6) reveals clear symptoms of network overfitting. The rapid degradation of validation results, observable around the 10th epoch, indicates a loss of the model's generalization capability. Furthermore, as early as the 4th epoch, a significant flattening of the validation loss curve can be observed, while the training loss continues to decrease. This points to a high likelihood of memorization, in which the model, rather than extracting universal facial expression patterns, begins to overfit to the specific characteristics of individual images in the training set. To mitigate this issue and improve the network's performance, the following optimization steps have been planned for subsequent stages of the work:

- Data Cleaning - A detailed review of the datasets will be conducted with respect to the correctness of annotations. Images of extremely low quality, those not depicting faces, or those carrying incorrect labels (noisy data) will be removed from the training process.
- Data Augmentation - The impact of random geometric and photometric image transformations applied in real time (e.g., rotations, brightness adjustments) on suppressing overfitting and improving the model's final evaluation metrics will be investigated.

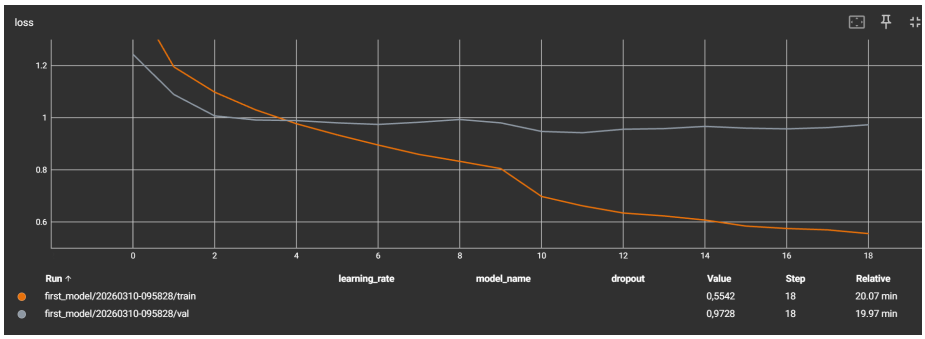

Figure 6: Graph of validation and training loss

4.4 Extended base model

References

[1] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, i M. Grundmann, „BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs", arXiv preprint arXiv:1907.05047, 2019. [Link to the article]

[2] C. Lugaresi et al., „MediaPipe: A Framework for Building Perception Pipelines", arXiv preprint arXiv:1906.08172, 2019. [Link to the article]

[3] M. Abadi et al., „TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems", arXiv preprint arXiv:1603.04467, 2016. [Link to the article]

[4] (fer2013+affectnet) dataset - Emotions - found on kaggle site form user PrasadSomvanshi on license Apache 2.0 link to dataset

[5] Facial Affect Dataset Balanced - found on kaggle site from user Viktor Modroczký on license Attribution-NonCommercial-ShareAlike 3.0 IGO (CC BY-NC-SA 3.0 IGO) link to the dataset

[6] Human Face Emotions - found on kaggle site from user Samith Chimminiyan on License Apache 2.0 link to the dataset

[7] Facial Emotion Recognition Dataset - found on kaggle site from user FahadullahA on license: Attribution 4.0 International (CC BY 4.0) link to the dataset

[8] C. Schuhmann et al., „EmoNet-Face: An Expert-Annotated Benchmark for Synthetic Emotion Recognition", arXiv preprint arXiv:2505.20033, 2025. [Link to the article]